

PÓS-GRADUAÇÃO FIAP — POSTECH

ARQUITETURA E DESENVOLVIMENTO DE SISTEMAS DE SOFTWARE

FIAP RESTAURANTE — FASE 3

Documentação de arquitetura do sistema de pedidos on-line para restaurante

Danilo Fernando de Paula e Silva

Gilmar da Costa Moraes Junior

Juliana Maria Dal Olio Braz

Luis Henrique Silveira Borges

Thiago de Jesus Cordeiro

São Paulo — Maio de 2026

Danilo Fernando de Paula e Silva
Gilmar da Costa Moraes Junior
Juliana Maria Dal Olio Braz
Luis Henrique Silveira Borges
Thiago de Jesus Cordeiro

FIAP RESTAURANTE — FASE 3

Documentação de arquitetura do sistema de pedidos on-line para restaurante

Documentação técnica apresentada ao programa de Pós-Graduação FIAP — PosTech como parte das entregas do Tech Challenge da Fase 3. O sistema descrito implementa um conjunto de microsserviços para gerenciamento de pedidos, integração com gateway externo de pagamento e fluxo de produção da cozinha.

São Paulo — Maio de 2026

RESUMO

Este documento descreve a arquitetura de um sistema distribuído desenvolvido como entrega do Tech Challenge da Fase 3 do programa de Pós-Graduação FIAP — PosTech. O sistema implementa o fluxo on-line de pedidos para um restaurante, incluindo cadastro e autenticação de clientes, criação e confirmação de pedidos, comunicação com um gateway externo eventualmente disponível para processamento de pagamentos, tratamento de falhas com resiliência e propagação do pedido para a cozinha após a aprovação do pagamento. A solução é composta de quatro microsserviços Spring Boot 4 / Java 25, comunicando-se de forma síncrona via gRPC e GraphQL e assíncrona via Apache Kafka, com persistência em MySQL 8.4 isolada por bounded context. A resiliência da chamada ao gateway é provida pela biblioteca Resilience4j (Circuit Breaker, Retry, Timeout e Fallback), e a recuperação automática é garantida por um worker agendado. Toda a orquestração local é provida por Docker Compose, permitindo execução do sistema completo com um único comando.

Palavras-chave: microsserviços; arquitetura hexagonal; Apache Kafka; Resilience4j; Spring Boot; GraphQL; Docker; JWT.

SUMÁRIO

RESUMO	3
SUMÁRIO	4
1 INTRODUÇÃO	7
2 VISÃO GERAL DO SISTEMA	7
2.1 Domínio do problema	7
2.2 Requisitos atendidos	8
2.3 Componentes externos	8
3 ARQUITETURA	8
3.1 Visão de componentes	8
3.2 Microserviços	9
3.2.1 <i>usuario-autenticacao</i>	9
3.2.2 <i>restaurante-pedido</i>	10
3.2.3 <i>pagamento</i>	10
3.2.4 <i>restaurante-service</i>	10
3.3 Infraestrutura	10
3.3.1 <i>Apache Kafka em modo KRaft</i>	10
3.3.2 <i>MySQL</i>	11
3.3.3 <i>procpag</i>	11
4 FLUXO PRINCIPAL	11
5 RESILIÊNCIA E TOLERÂNCIA A FALHAS	12
5.1 Padrões aplicados	12
5.2 Worker de reprocessamento	13
5.3 Garantias e limitações	13
6 SEGURANÇA	13
6.1 Autenticação e autorização	13
6.2 Ownership check em pedidos	14
7 DECISÕES ARQUITETURAIS	14
8 ESTRATÉGIA DE TESTES	15
9 OPERAÇÃO E DEPLOY	15

9.1 Limites de recursos	15
9.2 Observabilidade	16
9.3 Performance	16
10 CONSIDERAÇÕES FINAIS	16
REFERÊNCIAS	17

1 INTRODUÇÃO

Os sistemas de pedidos on-line para restaurantes evoluíram rapidamente nos últimos anos, pressionados pelo crescimento do delivery e pela necessidade de integração com gateways externos de pagamento, plataformas de logística e displays operacionais nas cozinhas. Esses sistemas precisam ser **resilientes** (toleram a indisponibilidade temporária de serviços de terceiros), **escaláveis** (suportam picos como horário de almoço sem degradar a experiência) e **seguros** (proteção dos dados do cliente e dos pagamentos).

Este documento apresenta a arquitetura do **FIAP Restaurante**, um sistema distribuído desenvolvido como Tech Challenge da Fase 3 do programa de Pós-Graduação FIAP — PosTech. O sistema cobre o ciclo completo de um pedido: cadastro do cliente, autenticação, criação e confirmação do pedido, comunicação com um gateway externo eventualmente disponível, tratamento de falhas, e propagação do pedido para a cozinha até sua finalização.

A solução é composta de quatro microsserviços Spring Boot 4 / Java 25, conectados por um broker Apache Kafka em modo KRaft e persistindo em databases isolados sobre uma instância MySQL 8.4. Toda a infraestrutura é descrita em um único arquivo **docker-compose.yml**, que sobe o sistema completo (oito containers) com um único comando.

Este documento detalha a arquitetura, as decisões técnicas que a guiaram, os mecanismos de resiliência implementados, a estratégia de testes adotada e os procedimentos de operação. Decisões pontuais estão também registradas em ADRs (Architecture Decision Records) na pasta docs/adr/ do repositório.

2 VISÃO GERAL DO SISTEMA

2.1 Domínio do problema

O sistema modela quatro **bounded contexts** da operação de um restaurante on-line:

- **Identidade do cliente:** cadastro, autenticação e emissão de tokens JWT.
- **Pedido:** criação, cálculo de valor total, confirmação, consulta e atualização de status conforme eventos externos.
- **Pagamento:** processamento da cobrança contra um gateway externo eventualmente disponível, com tratamento de falhas e reprocessamento automático.
- **Cozinha (produção):** fila de pedidos aprovados, com transições de estado disparadas pelo dono do restaurante (recebido, em preparo, pronto).

Cada um desses contextos foi materializado em um microsserviço independente, com modelo de dados próprio e fronteiras explícitas. A comunicação entre eles privilegia eventos assíncronos via Apache Kafka, recorrendo a chamadas síncronas (gRPC) apenas quando a operação exige.

2.2 Requisitos atendidos

Os requisitos formais da Fase 3 estão sumarizados a seguir, com indicação do componente responsável por cada um:

Req.	Descrição	Componente
4.1	Cadastro e autenticação de clientes	usuario-autenticacao
4.2	Criação e confirmação de pedido	restaurante-pedido
4.3	Consulta de pedido por ID e listagem por cliente	restaurante-pedido
4.4	Processamento de pagamento via gateway externo	pagamento + procpag
4.5	Marcação de pagamento pendente em falha	pagamento (Resilience4j)
4.6	Reprocessamento automático ao restabelecimento	pagamento (@Scheduled)
4.7	Atualização automática do status do pedido	restaurante-pedido (consumer)
5.1	Arquitetura em múltiplos serviços	4 microserviços
5.2	Spring Security + JWT (RS256)	todos os serviços de aplicação
5.3	Comunicação assíncrona via Kafka	6 tópicos
5.4	Resiliência (CB + Retry + Timeout + Fallback)	pagamento.ExternalPaymentClient
5.5	Clean / Hexagonal Architecture	todos os 4 módulos

Além do mínimo exigido, foi implementado o módulo opcional **restaurante-service** (item 5.1 da spec), responsável pelo bounded context da cozinha.

2.3 Componentes externos

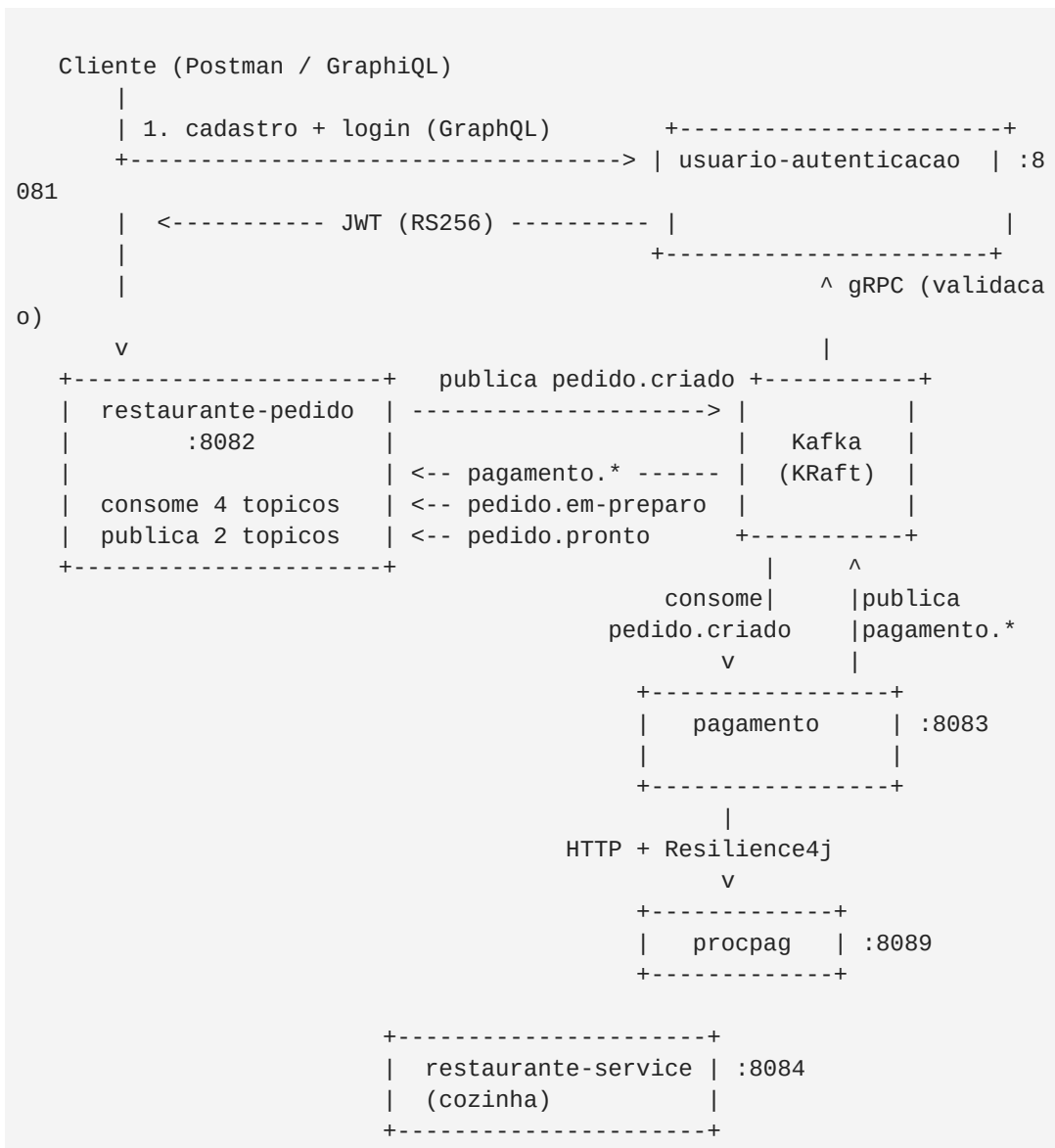
O sistema integra-se a um gateway externo de pagamento denominado **procpag**, fornecido pelos professores como uma imagem Docker (docker.io/erickemprobr/procpag). Esse serviço é deliberadamente instável: ora aprova, ora responde com erro ou timeout, simulando um gateway real eventualmente disponível. Toda a estratégia de resiliência da solução é desenhada em torno dessa imprevisibilidade.

3 ARQUITETURA

3.1 Visão de componentes

A arquitetura é composta de oito containers em rede privada Docker. Quatro deles são as aplicações Spring Boot (uma por bounded context), três são peças de infraestrutura (MySQL, Kafka e Kafka UI) e o oitavo é o gateway externo procpag.

A figura abaixo, em representação textual, descreve a topologia. O diagrama em formato Mermaid está disponível em `docs/diagramas/componentes.md`.



3.2 Microserviços

Cada microserviço expõe sua API pública via GraphQL (Spring GraphQL), com GraphiQL embutido na URL `/graphiql`. Schemas residem em `src/main/resources/graphql/schema.graphqls` de cada módulo.

3.2.1 usuario-autenticacao

Responsável por cadastro de usuários, autenticação, emissão de JWT (RS256) e consulta interna via gRPC. Persiste no database `auth_db`. Não consome nem publica eventos Kafka. Expõe servidor gRPC na porta interna 9000 para os demais serviços consultarem dados de usuário de forma síncrona.

3.2.2 restaurante-pedido

Responsável pelo ciclo de vida do pedido: criação, cálculo de valor total, confirmação, consulta por ID, listagem por cliente autenticado e atualização automática de status conforme eventos vindos do pagamento e da cozinha. Persiste no database `pedido_db`. É o componente que publica os eventos `pedido.criado` (disparando o pagamento) e `pedido.pronto-para-cozinha` (após confirmação do pagamento).

3.2.3 pagamento

Consome o evento `pedido.criado` e tenta processar a cobrança contra o gateway externo procpag. Aplica Resilience4j (Circuit Breaker, Retry, Timeout, Fallback). Em caso de falha, marca o pagamento como pendente e publica `pagamento.pendente`; em caso de sucesso, publica `pagamento.aprovado`. Hospeda também o worker `@Scheduled` que, a cada 30 segundos, drena os pagamentos pendentes tentando novamente a chamada ao gateway. Persiste no database `pagamento_db`.

3.2.4 restaurante-service

Modela o bounded context da cozinha. Consome `pedido.pronto-para-cozinha` e cria um agregado `PedidoCozinha` no estado RECEBIDO. O dono do restaurante (perfil `DONO_RESTAURANTE` no JWT) avança o estado via mutations GraphQL: `iniciarPreparo` (RECEBIDO → EM_PREPARO) e `marcarComoPronto` (EM_PREPARO → PRONTO). Cada transição publica um evento (`pedido.em-preparo` ou `pedido.pronto`) consumido pelo `restaurante-pedido` para refletir o status no agregado principal. Persiste no database `cozinha_db`.

3.3 Infraestrutura

3.3.1 Apache Kafka em modo KRaft

O broker roda em **modo KRaft** (Kafka Raft, GA desde Kafka 3.3), eliminando a dependência de Zookeeper. Em configuração **single-node** apropriada para desenvolvimento e demonstração, com replication factor 1 nos tópicos internos. O ADR 0006 detalha a decisão.

São **seis tópicos**orquestrando o fluxo:

Tópico	Publicador	Consumidor(es)
<code>pedido.criado</code>	<code>restaurante-pedido</code>	<code>pagamento</code>
<code>pagamento.aprovado</code>	<code>pagamento</code>	<code>restaurante-pedido</code>
<code>pagamento.pendente</code>	<code>pagamento</code>	<code>restaurante-pedido</code>
<code>pedido.pronto-para-cozinha</code>	<code>restaurante-pedido</code>	<code>restaurante-service</code>
<code>pedido.em-preparo</code>	<code>restaurante-service</code>	<code>restaurante-pedido</code>
<code>pedido.pronto</code>	<code>restaurante-service</code>	<code>restaurante-pedido</code>

A chave de cada mensagem é o `pedidoId.toString()`, garantindo que todos os eventos do mesmo pedido caem na mesma partição e preservam ordem por agregado.

3.3.2 MySQL

Uma única instância MySQL 8.4 hospeda **quatro databases** isolados, um por microserviço — abordagem *database per service*. O script `init.sql` cria os quatro databases na primeira inicialização do container. O ADR 0007 detalha a decisão de manter os bancos isolados, mas na mesma instância MySQL.

O MySQL recebe tuning explícito no docker-compose:

- `innodb-buffer-pool-size=512M` (default é 128 MB)
- `innodb-redo-log-capacity=128M`
- `innodb-flush-log-at-trx-commit=2` (latência menor; tolerância de até 1 segundo perdido em crash do host)
- `innodb-flush-method=O_DIRECT` (evita double-buffering)
- `skip-name-resolve` (sem reverse DNS por conexão)

3.3.3 procpag

Gateway de pagamento externo fornecido pelos professores (imagem `docker.io/erickemprobr/procpag`). Simula um serviço eventualmente disponível, alternando respostas de aprovação com erros e timeouts. Toda a estratégia de resiliência (capítulo 5) gira em torno dessa imprevisibilidade controlada.

4 FLUXO PRINCIPAL

O fluxo feliz (sem falhas) do sistema percorre, em ordem, os passos descritos a seguir. O diagrama de sequência em `docs/diagramas/sequencia-happy-path.md` apresenta a mesma narrativa em forma gráfica.

1. O cliente envia `mutation login` ao `usuario-autenticacao`, que valida credenciais e devolve um JWT RS256 contendo nas claims o `subject` (UUID do cliente) e a `claim groups` com seu perfil.

2. Com o token no header `Authorization: Bearer <jwt>`, o cliente envia `mutation criarPedido` ao `restaurante-pedido`. O serviço calcula o valor total a partir dos itens, persiste o pedido no status **CRIADO** e devolve o agregado completo. O `clienteId` é sempre extraído do JWT, nunca recebido do cliente (req. 5.2).

3. O cliente envia `mutation confirmarPedido`. O status passa para **CONFIRMADO** e o evento `pedido.criado` é publicado no Kafka. O cliente recebe a resposta em milissegundos.

4. O *pagamento* consome `pedido.criado`, persiste um registro de pagamento e chama o *procpag* via HTTP. A chamada é envolvida por `Resilience4j` (CB + Retry + Timeout). No caminho feliz, o *procpag* responde 200 OK.

5. O *pagamento* persiste o pagamento como **APROVADO** e publica `pagamento.aprovado`.

6. O *restaurante-pedido* consome o evento, transita o pedido para **PAGO** e publica `pedido.pronto-para-cozinha` com os itens (sem preço — irrelevante para a cozinha).

7. O *restaurante-service* consome `pedido.pronto-para-cozinha` e cria o agregado `PedidoCozinha` no estado **RECEBIDO**.

8. O dono do restaurante consulta a fila via query `filaCozinha` em `:8084/graphql` e aciona mutation `iniciarPreparo`. O agregado passa a **EM_PREPARO**; o evento `pedido.em-preparo` é publicado.

9. O *restaurante-pedido* consome o evento e atualiza o pedido principal para **EM_PREPARO**. Quando o preparo termina, o dono aciona `marcarComoPronto`, o evento `pedido.pronto` propaga e o ciclo se fecha com o pedido em **PRONTO**.

5 RESILIÊNCIA E TOLERÂNCIA A FALHAS

A resiliência do sistema concentra-se na integração com o gateway externo *procpag*, ponto único de instabilidade previsível. Toda a estratégia foi implementada com **Resilience4j 2.3.0** via anotações no método de integração HTTP, e complementada por um worker agendado para reprocessamento automático.

5.1 Padrões aplicados

Os quatro padrões clássicos de resiliência foram aplicados em camadas:

- **Retry** — três tentativas com backoff exponencial (2s, 4s, 8s).
- **Timeout** — cinco segundos por chamada; chamadas mais lentas são interrompidas.
- **Circuit Breaker** — abre após 50% de falhas em uma janela deslizante de 5 chamadas. Quando aberto, novas chamadas falham imediatamente sem tocar o gateway. Após 30 segundos, transita para **HALF_OPEN** e testa 2 chamadas; se passarem, fecha.
- **Fallback** — quando todas as estratégias acima falham, o pagamento é marcado como **PENDENTE**, e o evento `pagamento.pendente` é publicado no Kafka. O pedido nunca falha do ponto de vista do cliente.

5.2 Worker de reprocessamento

Vive exclusivamente no módulo *pagamento*. É um `@Scheduled` que executa a cada 30 segundos (configurável em `pagamento.reprocess.fixed-delay-ms`), busca um lote de até 20 pagamentos no estado PENDENTE e retenta cada um deles contra o *procpag*. Quando o gateway voltar e a chamada for bem-sucedida, o pagamento é marcado APROVADO e `pagamento.aprovado` é publicado — fechando o ciclo de recuperação.

Por que o worker está no pagamento, e não no restaurante-pedido? O worker opera sobre dados do próprio bounded context (registros de pagamento). Reprocessar significa tentar de novo a chamada ao gateway, integração que vive no pagamento e em mais lugar nenhum. Manter o worker fora do *restaurante-pedido* preserva a separação de responsabilidades: o *restaurante-pedido* apenas reage a eventos.

5.3 Garantias e limitações

O sistema garante **at-least-once** na entrega de eventos Kafka (o consumer pode receber o mesmo evento mais de uma vez em cenários de reprocessamento). As transições de estado nas entidades de domínio são **idempotentes**: marcar um pedido já PAGO como PAGO novamente não causa efeito. Isso é exercitado nos testes unitários do agregado *Pedido*.

Não há, no escopo atual, mecanismo de **outbox** ou transações distribuídas entre o commit do banco local e a publicação no Kafka. Em cenário de crash entre as duas operações, há risco teórico de inconsistência (o pedido vira PAGO no banco mas o evento não é publicado). Para o escopo do projeto, esse risco é considerado aceitável.

6 SEGURANÇA

6.1 Autenticação e autorização

A autenticação é baseada em **JWT RS256** emitido pelo *usuario-autenticacao*. A chave privada RSA vive apenas neste serviço; a chave pública é distribuída no classpath dos demais (`src/main/resources/keys/publicKey.pem`), permitindo validação local sem necessidade de chamar o *usuario-autenticacao* a cada requisição. O ADR 0008 detalha a escolha pela assinatura assimétrica.

A autorização usa **perfis** codificados na `claim groups` do JWT: USUARIO (cliente final) e DONO_RESTAURANTE (administrador da fila da cozinha). Cada resolver GraphQL é protegido por `@PreAuthorize`, com regras específicas para cada operação.

6.2 Ownership check em pedidos

Na query `pedidoPorId`, o serviço extrai o `clienteId` do JWT e compara com o do pedido buscado. Pedidos de outros clientes retornam `null` — a mesma resposta de 'não encontrado', para não vazar a existência do recurso. Esta verificação foi adicionada como melhoria de segurança identificada em auditoria de conformidade.

7 DECISÕES ARQUITETURAIS

Cada decisão técnica significativa do projeto está registrada como ADR (Architecture Decision Record) na pasta `docs/adr/` do repositório, no formato proposto por Michael Nygard (Contexto + Decisão + Consequências). O índice completo está em `docs/adr/README.md`.

São 13 ADRs vigentes:

ADR	Decisão	Status
0001	Adoção de microsserviços em vez de monolito	Accepted
0002	Arquitetura hexagonal em cada microsserviço	Accepted
0003	GraphQL no ponto de entrada das aplicações	Accepted
0004	gRPC para chamadas síncronas entre serviços	Accepted
0005	Apache Kafka para comunicação assíncrona	Accepted
0006	Kafka em modo KRaft (sem Zookeeper)	Accepted
0007	Database separado por serviço (mesmo MySQL)	Accepted
0008	JWT com assinatura assimétrica RS256	Accepted
0009	Resilience4j para resiliência na integração	Accepted
0010	Bounded context separado para a cozinha	Accepted
0011	Docker Compose como orquestrador	Accepted
0012	AppCDS + Layered JARs no Dockerfile	Accepted
0013	Resource limits explícitos no compose	Accepted

Os ADRs descrevem não apenas a decisão tomada mas também as **alternativas consideradas e descartadas**, preservando o raciocínio que conduziu à escolha. Quando uma decisão for revisitada no futuro, o ADR existente receberá status *Superseded by ADR-XXXX* e um novo ADR será criado.

8 ESTRATÉGIA DE TESTES

A suíte de testes do projeto totaliza **285 testes automatizados** distribuídos pelos quatro módulos, executando em aproximadamente três minutos. Toda a suíte roda sem dependências externas — utiliza H2 em memória no lugar do MySQL, `@EmbeddedKafka` no lugar do broker real e mocks para integrações HTTP.

Módulo	Testes	Cobertura
usuario-autenticacao	63	Domain, use cases, JWT, gRPC, GraphQL, smoke
restaurante-pedido	126	Domain, 4 consumers, 2 publishers, GraphQL, smoke
pagamento	57	Domain, Resilience4j, worker, GraphQL, smoke
restaurante-service	39	Domain, 4 use cases, JPA, Kafka, GraphQL, smoke
TOTAL	285	

A pirâmide de testes adotada privilegia **testes unitários** rápidos (JUnit 5 + Mockito + AssertJ) para a maior parte da cobertura, complementados por **smoke tests** de contexto Spring (`@SpringBootTest`) e por **testes de integração** da camada GraphQL via MockMvc. O domínio puro permite que regras de negócio sejam testadas sem qualquer dependência de framework.

9 OPERAÇÃO E DEPLOY

Todo o sistema é orquestrado por Docker Compose. O comando único de subida é:

```
docker compose up -d --build
```

Esse comando constrói as imagens das quatro aplicações (quando necessário) e sobe os oito containers. O primeiro build é demorado porque cada Dockerfile executa um *training run* de AppCDS para gerar o archive de classes compartilhadas (ver ADR 0012). Execuções subsequentes aproveitam o cache de camadas e ficam em poucos segundos.

9.1 Limites de recursos

Cada serviço tem limites explícitos de memória e CPU configurados em `deploy.resources.limits`. O total combinado fica em aproximadamente 3,5 GB de RAM e 7,5 vCPUs, permitindo execução em máquinas convencionais sem comprometer o sistema operacional do host. O ADR 0013 detalha a calibração.

9.2 Observabilidade

Cada aplicação expõe Spring Boot Actuator nos endpoints `/actuator/health`, `/actuator/info` e `/actuator/metrics`. O serviço *pagamento* expõe adicionalmente `/actuator/circuitbreakers` (estado dos breakers Resilience4j), `/actuator/retries` e `/actuator/scheduledtasks` (confirma o worker ativo). Os tópicos Kafka podem ser inspecionados via Kafka UI em `http://localhost:8085`.

9.3 Performance

Três otimizações foram aplicadas para reduzir o consumo de recursos e acelerar o startup das aplicações Spring:

- **Kafka em modo KRaft** — eliminou o container Zookeeper, reduzindo em ~150 MB o consumo de RAM (ADR 0006).
- **AppCDS** — pré-resolve classes em um archive binário gerado durante o docker build, reduzindo o boot da JVM em aproximadamente 30 por cento (ADR 0012).
- **Layered JARs** — o jar é particionado em camadas semânticas, permitindo cache mais inteligente do Docker em rebuilds: mudança de código invalida apenas a camada de aplicação (~5 MB), não o jar completo (~50 MB) (ADR 0012).

10 CONSIDERAÇÕES FINAIS

A arquitetura desenhada cumpre integralmente os requisitos da Fase 3 e ainda implementa o módulo opcional *restaurante-service*, expandindo o fluxo do pedido até a entrega pela cozinha. Os principais ganhos da abordagem distribuída — isolamento de falhas, escalabilidade granular, evolução independente — são contrabalançados pelo aumento da complexidade operacional, que se manifesta em oito containers, quatro databases e um broker de mensageria coordenando tudo.

Decisões pontuais foram documentadas em ADRs para preservar o raciocínio que conduziu a cada escolha, permitindo que evoluções futuras tenham contexto completo. Diagramas em formato Mermaid acompanham esta documentação na pasta `docs/diagramas/`.

Como pendência identificada e não resolvida no escopo da Fase 3, registra-se a ausência de mecanismo de **outbox** para garantir atomicidade entre commit do banco local e publicação de eventos Kafka, e a não implementação de **correlation IDs** para rastreabilidade distribuída de requisições — itens naturais para uma próxima iteração.

O sistema está pronto para demonstração, com suíte verde de 285 testes e roteiro de apresentação detalhado em `docs/roteiro-video.md`.

REFERÊNCIAS

COCKBURN, Alistair. **Hexagonal Architecture**. 2005. Disponível em: <https://alistair.cockburn.us/hexagonal-architecture/>. Acesso em: 25 May. 2026.

EVANS, Eric. **Domain-Driven Design: Tackling Complexity in the Heart of Software**. Boston: Addison-Wesley, 2003.

NYGARD, Michael. **Documenting Architecture Decisions**. 2011. Disponível em: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>. Acesso em: 25 May. 2026.

PIVOTAL SOFTWARE. **Spring Boot Reference Documentation**. 2025. Disponível em: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>. Acesso em: 25 May. 2026.

APACHE SOFTWARE FOUNDATION. **Apache Kafka Documentation**. 2025. Disponível em: <https://kafka.apache.org/documentation/>. Acesso em: 25 May. 2026.

RESILIENCE4J. **Resilience4j User Guide**. 2025. Disponível em: <https://resilience4j.readme.io/>. Acesso em: 25 May. 2026.

FOWLER, Martin. **Microservices**. 2014. Disponível em: <https://martinfowler.com/articles/microservices.html>. Acesso em: 25 May. 2026.

OPENJDK. **JEP 310: Application Class-Data Sharing**. 2018. Disponível em: <https://openjdk.org/jeps/310>. Acesso em: 25 May. 2026.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação — trabalhos acadêmicos — apresentação. Rio de Janeiro, 2011.

FIAP. **Tech Challenge — Fase 3**. Material institucional do programa de Pós-Graduação PosTech. São Paulo, 2025.